МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №3
Варіант 4

на тему «Незмінні структури даних (immutability) у Python»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. застосувати tuple, frozenset, dataclasses(frozen=True), MappingProxyType; замінити мутації на pure updates; виміряти вплив незмінності на тести та час виконання.

Теоретичні відомості

Функціональний стиль спирається на чисті функції та **незмінні структури**. Незмінність дає референтну прозорість, просту композицію, безпечне багатопотокове читання і можливість кешування (хешовані структури можуть бути ключами dict / set).

tuple замінює list, **frozenset** — set, **@dataclass(frozen=True, slots=True)** — звичайні класи (FrozenInstanceError при спробі зміни, автоматичні __eq__ і __hash__), **MappingProxyType** — read-only view над dict (джерело слід копіювати).

Оновлення виконуються як **pure updates** — повернення нової копії через dataclasses.replace, конкатенацію tuple або операції frozenset.

Індивідуальне завдання (варіант 4)

Тема «Склад товарів». Незмінна модель: Product, StockRecord, tags, supplier meta. Pure-оновлення: поповнення залишку, зміна ціни, додавання тегу. Кожна операція повертає новий StockRecord.

Хід роботи

1. Незмінна модель

Product і StockRecord — frozen dataclass зі slots; історія рухів — tuple з NamedTuple Movement; теги — frozenset; метадані постачальника — MappingProxyType, створена з копії словника. Поле supplier виключено з hash (MappingProxyType не хешується).

```
def make_meta(**kwargs: str) -> Mapping[str, str]:
    """Read-only метадані. Спочатку робимо КОПІЮ, щоб зовні ніхто не тримав
    посилання на «живий» dict під proxy."""
    return MappingProxyType(dict(kwargs))

@dataclass(frozen=True, slots=True)
class StockRecord:
    product: Product
    quantity: int
    tags: frozenset[str] = frozenset()
    # MappingProxyType не хешується, тому поле бере участь у ==, але НЕ в hash.
    supplier: Mapping[str, str] = field(default_factory=lambda: make_meta(), hash=False)
    history: tuple[Movement, ...] = ()

    def __post_init__(self) -> None:
        if self.quantity < 0:
            raise ValueError("quantity must be non-negative")
```

```
    # --- обчислення (чисті методи) -----------------------------------
    def value(self) -> float:
        """Вартість залишку: ціна * кількість."""
        return self.product.price * self.quantity

    def needs_reorder(self, min_stock: int) -> bool:
        return self.quantity < min_stock

    # --- pure updates (повертають НОВИЙ об'єкт) ------------------------
    def with_quantity(self, quantity: int, movement: Movement) -> StockRecord:
        return replace(self, quantity=quantity, history=self.history + (movement,))
    def with_tag(self, tag: str) -> StockRecord:
        return replace(self, tags=self.tags | frozenset({tag}))
```

Лістинг 1 — фрагмент файлу model.py

2. Pure updates

Кожна функція повертає новий StockRecord, оригінал лишається незмінним.

```
def restock(record: StockRecord, qty: int, note: str = "") -> StockRecord:
    """Поповнення залишку."""
    if qty <= 0:
        raise ValueError("restock quantity must be positive")
    return record.with_quantity(record.quantity + qty, Movement("in", qty, note))
def change_price(record: StockRecord, new_price: float) -> StockRecord:
    """Зміна ціни: новий Product усередині нового StockRecord."""
    new_product = replace(record.product, price=new_price)  # валідація у Product
    movement = Movement("price", 0, f"{record.product.price} -> {new_price}")
    return replace(record, product=new_product, history=record.history + (movement,))
def add_tag(record: StockRecord, tag: str) -> StockRecord:
    """Додавання тегу (frozenset |)."""
    clean = tag.strip().lower()
    if not clean:
        raise ValueError("tag must not be empty")
    return record.with_tag(clean)
def update_supplier(record: StockRecord, **changes: str) -> StockRecord:
    """Нові метадані постачальника = копія старих + зміни, знову read-only."""
    return replace(record, supplier=make_meta(**{**record.supplier, **changes}))

def apply_clearance_discount(record: StockRecord, percent: float) -> StockRecord:
    """Правило знижки як чиста функція: лише для товарів з тегом ``clearance``."""
    if not 0 <= percent < 1:
        raise ValueError("percent must be in [0, 1)")
    if "clearance" not in record.tags:
        return record
    return change_price(record, round(record.product.price * (1 - percent), 2))
```

Лістинг 2 — фрагмент файлу core.py

3. Кешування та глибоке заморожування

Оскільки StockRecord хешований, tuple записів може бути ключем lru_cache. Утиліта freeze рекурсивно перетворює dict → MappingProxyType, set → frozenset, list → tuple.

```
@lru_cache(maxsize=128)
def total_inventory_value(records: tuple[StockRecord, ...]) -> float:
    """Кешована чиста функція: tuple з хешованих StockRecord — валідний ключ."""
    return round(sum(r.value() for r in records), 2)

def freeze(obj: Any) -> Any:
```

```
    """Рекурсивно: dict -> MappingProxyType, set -> frozenset, list -> tuple."""
    if isinstance(obj, Mapping):
        return MappingProxyType({k: freeze(v) for k, v in obj.items()})
    if isinstance(obj, (set, frozenset)):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj
```

Лістинг 3 — фрагмент файлу core.py

4. Тести немутації, read-only та хешованості

Написано 21 тест: заборона зміни полів, типи колекцій, оригінал після кожного оновлення, TypeError при записі в proxy, копіювання джерела meta, хешованість, правило знижки, lru_cache, freeze, а також порівняння часу з мутабельною версією baseline.py.

```
def test_restock_returns_new_copy(record: StockRecord) -> None:
    snapshot = make_record()
    new = restock(record, 10, "поставка")
    assert new is not record
    assert record == snapshot  # оригінал без змін
    assert new.quantity == 15
    assert new.history == (Movement("in", 10, "поставка"),)
    assert new.value() == record.value() + 10 * 1000.0

def test_hashability(record: StockRecord) -> None:
    records = {record, restock(record, 1), make_record()}
    assert len(records) == 2  # копія з тим самим вмістом == оригінал
    assert record is not make_record()
    assert hash(record) == hash(make_record())
    assert index_by_record([record])[record] == "EL-002"
```

Лістинг 4 — фрагмент файлу tests/test_immutability.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
=== Pure updates: кожна операція повертає НОВИЙ StockRecord ===
Початковий запис:      EL-002 Клавіатура     ціна= 1350.00 к-ть=   3 теги=[clearance]
Після restock(+20):    EL-002 Клавіатура     ціна= 1350.00 к-ть=  23 теги=[clearance]
Після change_price:    EL-002 Клавіатура     ціна= 1500.00 к-ть=  23 теги=[clearance]
Після add_tag:         EL-002 Клавіатура     ціна= 1500.00 к-ть=  23 теги=[clearance, promo]
Після знижки 20%:      EL-002 Клавіатура     ціна= 1200.00 к-ть=  23 теги=[clearance, promo]
Оригінал (без змін):   EL-002 Клавіатура     ціна= 1350.00 к-ть=   3 теги=[clearance]
keyboard is v4: False | keyboard == v4: False
Історія v4: [('in', 20, 'поставка №17'), ('price', 0, '1350.0 -> 1500.0'), ('price', 0,
    '1500.0 -> 1200.0')]
=== Read-only метадані постачальника ===
Постачальник: ТОВ Техно {'name': 'ТОВ Техно', 'country': 'UA', 'phone': '+380441112233'}
Спроба змінити meta -> TypeError : 'mappingproxy' object does not support item assignment
Нові meta: {'name': 'ТОВ Техно', 'country': 'UA', 'phone': '+380500000000'}
Старі meta: {'name': 'ТОВ Техно', 'country': 'UA', 'phone': '+380441112233'}
=== Хешованість ===
StockRecord як ключ dict: ['v0', 'v4']
hash(v0) != hash(v4): True

=== Запити над складом ===
Потрібне дозамовлення (< 5 шт.): ['EL-002', 'OF-011']
Top-2 за вартістю: ['EL-001', 'OF-010']
Вартість складу: 139210.0
Вартість складу (з кешу): 139210.0
lru_cache: CacheInfo(hits=1, misses=1, maxsize=128, currsize=1)
```

Рисунок 1 — результат роботи програми (Lab_03)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_03
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 21 items
tests/test_immutability.py::test_frozen_fields_cannot_be_assigned PASSED [  4%]
tests/test_immutability.py::test_collection_types_are_immutable PASSED   [  9%]
tests/test_immutability.py::test_slots_no_dict PASSED                    [ 14%]
tests/test_immutability.py::test_mappingproxy_is_read_only PASSED        [ 19%]
tests/test_immutability.py::test_make_meta_copies_source PASSED          [ 23%]
tests/test_immutability.py::test_validation PASSED                       [ 28%]
tests/test_immutability.py::test_restock_returns_new_copy PASSED         [ 33%]
tests/test_immutability.py::test_ship PASSED                             [ 38%]
tests/test_immutability.py::test_change_price_creates_new_product PASSED [ 42%]
tests/test_immutability.py::test_add_and_remove_tag PASSED               [ 47%]
tests/test_immutability.py::test_update_supplier PASSED                  [ 52%]
tests/test_immutability.py::test_clearance_discount_rule PASSED          [ 57%]
tests/test_immutability.py::test_chain_of_updates_keeps_every_version PASSED [ 61%]
tests/test_immutability.py::test_hashability PASSED                      [ 66%]
tests/test_immutability.py::test_mappingproxy_itself_is_not_hashable PASSED [ 71%]
tests/test_immutability.py::test_queries PASSED                          [ 76%]
tests/test_immutability.py::test_lru_cache_on_pure_function PASSED       [ 80%]
tests/test_immutability.py::test_freeze_deep PASSED                      [ 85%]
tests/test_performance.py::test_compare_mutable_vs_immutable_updates PASSED [ 90%]
tests/test_performance.py::test_immutable_large_batch PASSED             [ 95%]
tests/test_performance.py::test_mutable_large_batch PASSED               [100%]
============================= 21 passed in 0.18s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_03)

Порівняння часу «до / після» (pytest -q -s --durations=5):

```
...................
mutable   : 0.0069 s / 5000 iterations
immutable : 0.0640 s / 5000 iterations
ratio     : x9.25
...
============================= slowest 5 durations =============================
0.09s call     tests/test_performance.py::test_immutable_large_batch
0.07s call     tests/test_performance.py::test_compare_mutable_vs_immutable_updates

(3 durations < 0.005s hidden.  Use -vv to show these durations.)
21 passed in 0.25s
```

Рисунок 3 — порівняння часу «до / після» (pytest -q -s --durations=5)

Висновок

Модель складу переведено на незмінні структури: frozen dataclass, tuple, frozenset і MappingProxyType. Усі зміни стану реалізовано як pure updates, тому кожна стара версія запису залишається валідною. Вимірювання показали, що незмінна модель повільніша на мікробенчмарку (кожне оновлення створює новий об'єкт), але абсолютна різниця становить десятки мікросекунд на операцію, а всі тести виконуються менш ніж за 0.3 с. Натомість отримано передбачуваний стан, хешованість і можливість кешування чистих функцій.